

Software Architecture and Design Specification

Project: Online Bookstore (Team 2, Section C)

Version: 1.0

Authors: Chenchu Sreekrishna Koushik, Deeptha S, G H Pramod, Hardhick M Gowda

Date: 05-10-2026

Status: Baselined for Sprint 1

Revision History

Version	Date	Author	Change Summary
0.5	02-10-2026	Group 2	Components, diagrams and API drafted.
1.0	04-10-2026	Group 2	Security architecture and traceability added; baselined.

Approvals

Role	Name	Signature/Date
Architecture and integration lead	Hardhick M Gowda	
Security lead	Chenchu Sreekrishna Koushik	
Catalogue and performance lead	Deeptha S	
Orders and payment lead	G H Pramod	

1. Introduction

1.1 Purpose

This document describes how the Online Bookstore is built: its parts, how they connect, and how the main features work.

1.2 Scope

The whole system: the React website, the Node.js/Express server, the MongoDB database, and the outside services it uses (payment gateway sandbox, e-mail, Redis cache, image storage).

1.3 Audience

The four developers of Group 2, testers, and the course faculty.

1.4 Definitions

API - the server's web interface. JWT - a signed login token. RBAC - permissions by role (Buyer, Seller, Admin). STRIDE - a checklist of six kinds of security threat. Webhook - a message the payment gateway sends to our server when a payment finishes.

2. Document Overview

2.1 How to use this document

Section 3 explains the architecture (the big picture). Section 4 explains the design (how the important parts work in detail). Requirement IDs (FR, NFR, SR) are the same as in the SRS.

2.2 Related Documents

SRS v1.1, Software Test Plan v1.0, Requirement Traceability Matrix (SRS Appendix C).

3. Architecture

3.1 Goals & Constraints

Goals:

- Search results in under 2 seconds for 200 users at once (NFR-01).
- Stock and payments always match - a book can never be sold twice (NFR-05).
- Secure by default: every request checked, no card data stored (SR-01 to SR-12).
- Easy for four students to build and test in one semester.

Constraints: MERN stack only; payments in sandbox mode only; free-tier hosting; one semester.

3.2 Stakeholders & Concerns

Stakeholder	Main concern
Buyer	Find books fast, pay safely, keep personal data private
Seller	List books quickly, keep stock correct, get paid
Administrator	Moderate content and see platform reports
Developers	Clear modules so four people can work in parallel
Faculty	Clear links from requirements to design and tests

3.3 Component (UML) Diagram

Figure 1 shows the main components. A small circle is an interface a component offers; a dashed arrow means "uses".

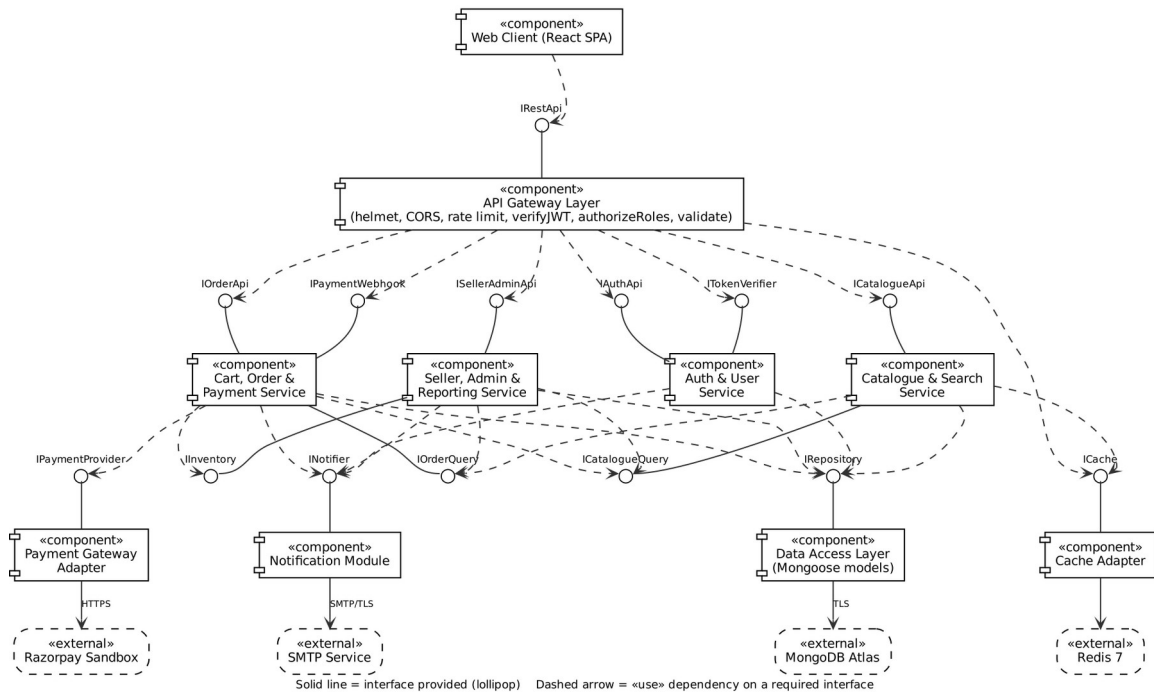


Figure 1: UML component diagram

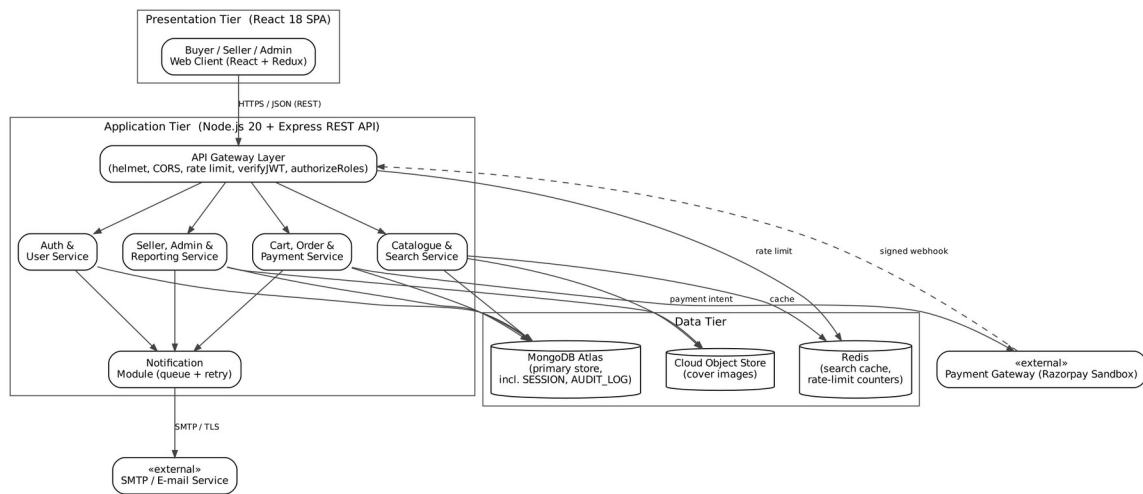


Figure 2: The three layers - website, server, data

3.4 Component Descriptions

Component	What it does	Owner
Web Client	React website used by buyers, sellers and admins.	All
API Gateway Layer	Runs on every request first: security headers, rate limit, login check, role check, input check.	Hardhick M Gowda
Auth & User Service	Register, OTP, login/logout, password reset, profile, addresses.	Chenchu Sreekrishna Koushik

Component	What it does	Owner
Catalogue & Search Service	Book list, search, filters, book details, reviews.	Deeptha S
Cart, Order & Payment Service	Cart, wishlist, checkout, payment, invoices, order tracking.	G H Pramod
Seller, Admin & Reporting Service	Listings, stock, shipping, seller dashboard, moderation, reports.	Hardhick M Gowda
Notification Module	Sends e-mails and retries if sending fails.	Chenchu Sreekrishna Koushik
Data Access Layer	Database models for all 17 collections.	All
Cache / Payment adapters	Small wrappers around Redis and the payment gateway, so they can be replaced in tests.	Deeptha S / G H Pramod

3.5 Chosen Architecture Pattern and Rationale

Pattern: layered (three-tier) architecture. The website, the server and the database are separate layers. Inside the server, the code is split into four modules - one per team member - and each module is layered again into routes, controllers, services and models.

Why:

- Each person owns one module, so work does not overlap.
- Placing an order changes the order, the payment and the stock together; in one server and one database this is a single safe transaction.
- One server fits the free hosting tier and is simple to deploy and test.

Rejected: Microservices (too complex: four servers to run and no single transaction for checkout). An unstructured app with no layers (hard to test and to divide between four people).

3.6 Technology Stack & Data Stores

Layer	Technology
Website	React 18, React Router, Axios, Vite
Server	Node.js 20, Express 4, Joi (input checks), bcrypt, JSON Web Tokens, helmet
Database	MongoDB 7 on Atlas (Mongoose) - 17 collections, see SRS Appendix B
Cache	Redis (search results for 5 minutes, login rate limits)
Outside services	Razorpay sandbox (payments), SMTP (e-mail), object store (cover images)
Testing and CI	Jest, Supertest, Playwright, k6, OWASP ZAP, GitHub Actions

3.7 Risks & Mitigations

Risk	Mitigation
Free-tier limits run out during testing	Keep seed data to 10,000 listings; store images outside the database
Payment sandbox is down	Use a fake (stub) gateway in tests
Two buyers buy the last copy at the same time	Reduce stock only if enough is left, inside one transaction
Search is too slow	Text index on books plus Redis cache; load test every sprint
Passwords or keys leak through GitHub	Keep secrets in .env only; secret scanning in CI

3.8 Traceability to Requirements

Every requirement maps to one component and at least one test case in the Test Plan.

Requirement	Component	Test case
FR-01	Auth & User Service	TC-A-01
FR-02	Auth & User Service	TC-A-02
FR-03	Auth & User Service	TC-A-03
FR-04	Auth & User Service	TC-A-04
FR-05	Auth & User Service	TC-A-05
FR-06	API Gateway Layer (role check)	TC-A-06
FR-07	Catalogue & Search Service	TC-C-01
FR-08	Catalogue & Search Service	TC-C-02
FR-09	Catalogue & Search Service	TC-C-03
FR-10	Catalogue & Search Service	TC-C-04
FR-11	Catalogue & Search Service	TC-C-05
FR-12	Catalogue & Search Service	TC-C-06
FR-13	Cart, Order & Payment Service	TC-O-01
FR-14	Cart, Order & Payment Service	TC-O-02
FR-15	Cart, Order & Payment Service	TC-O-03
FR-16	Cart, Order & Payment Service	TC-O-04
FR-17	Cart, Order & Payment Service	TC-O-05
FR-18	Cart, Order & Payment Service	TC-O-06
FR-19	Seller, Admin & Reporting Service	TC-S-01
FR-20	Seller, Admin & Reporting Service	TC-S-02
FR-21	Seller, Admin & Reporting Service	TC-S-03

Requirement	Component	Test case
FR-22	Seller, Admin & Reporting Service	TC-S-04
FR-23	Seller, Admin & Reporting Service	TC-S-05
FR-24	Seller, Admin & Reporting Service	TC-S-06
NFR-01	Catalogue & Search Service, Cache	TC-N-01
NFR-02	Deployment, /health endpoint	TC-N-02
NFR-03	Auth & User Service	TC-N-03
NFR-04	Web Client	TC-N-04
NFR-05	Cart, Order & Payment Service	TC-N-05
NFR-06	CI pipeline	TC-N-06
NFR-07	Web Client	TC-N-07
NFR-08	All services (audit log)	TC-N-08
SR-01 to SR-12	API Gateway Layer and services (section 3.9)	TC-SEC-01 to TC-SEC-10

3.9 Security Architecture

Every request passes the API Gateway Layer before it reaches any service. Passwords are stored only as bcrypt hashes. After login the user gets a 15-minute access token and a 7-day refresh token kept in a secure cookie. Card details go straight to the payment gateway and never touch our server.

Threat model (STRIDE):

Threat	Example	How we stop it	Req.
Spoofing	Guessing passwords; fake payment message	Login rate limit; OTP-verified accounts; check the gateway's signature	SR-01, SR-08, SR-10
Tampering	Changing the price in the request	Server always recalculates prices; input checks	SR-07
Repudiation	Admin denies blocking a seller	Audit log that can only be added to	SR-09
Information disclosure	Opening another user's order by its ID	Ownership check on every request; no passwords in logs	SR-03, SR-06
Denial of service	Flooding search or OTP e-mails	Rate limits; max 50 results per page	SR-10
Elevation of privilege	Buyer calling admin pages	Role check on every route; no self-registration as admin	SR-05

4. Design

4.1 Design Overview

Each server module has the same four layers:

1. **Route** - the URL and who may call it.
2. **Controller** - reads the request and sends the response.
3. **Service** - the business rules (for example, "price cannot be above MRP").
4. **Model** - the database collection.

Money is stored in paise (whole numbers) to avoid rounding errors. Order numbers look like OB-20261004-00001.

4.2 UML Sequence Diagrams

4.2.1 Register, verify e-mail and log in (FR-01 to FR-03)

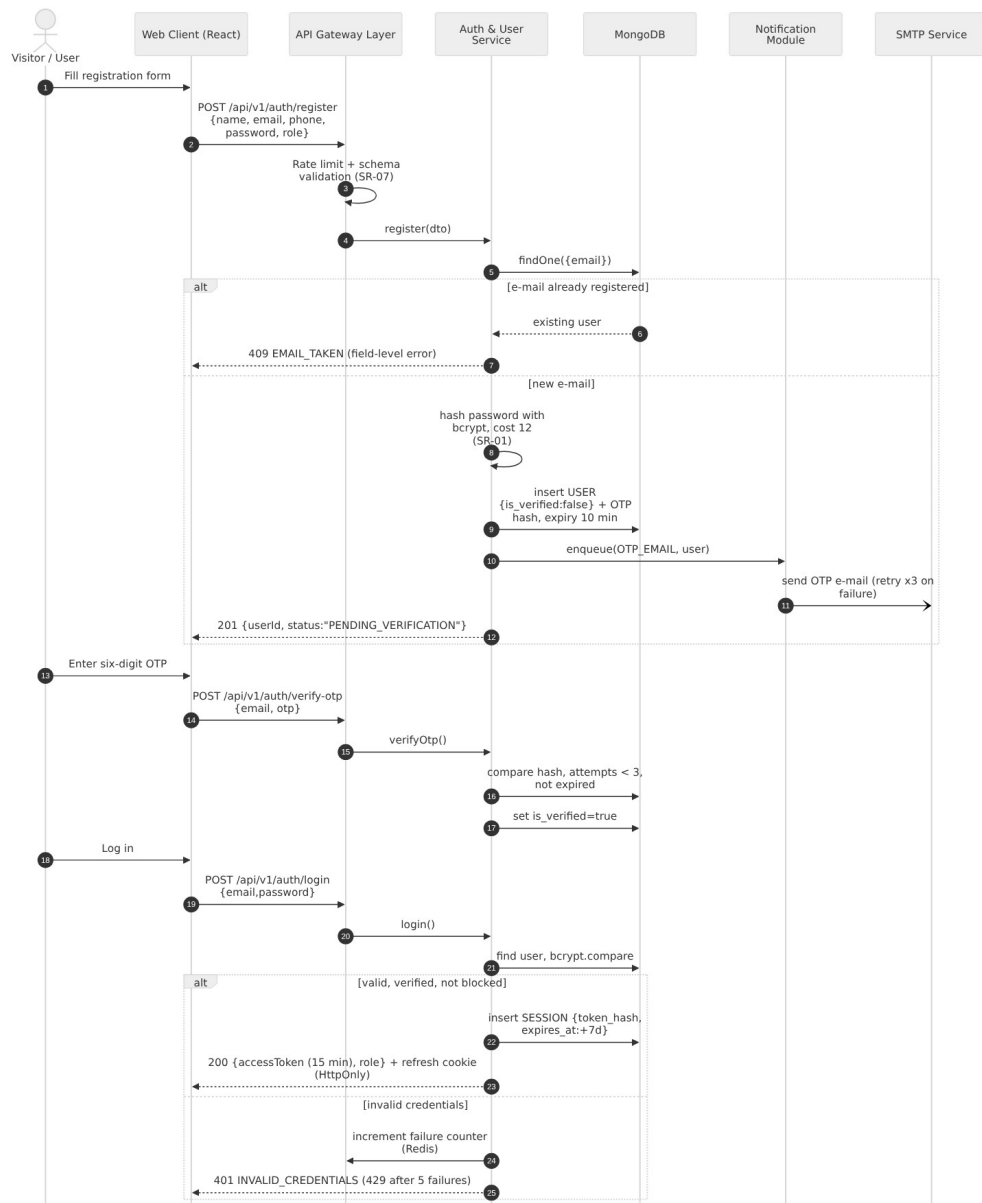


Figure 3: Registration, OTP and login

4.2.2 Checkout and payment (FR-15, FR-16)

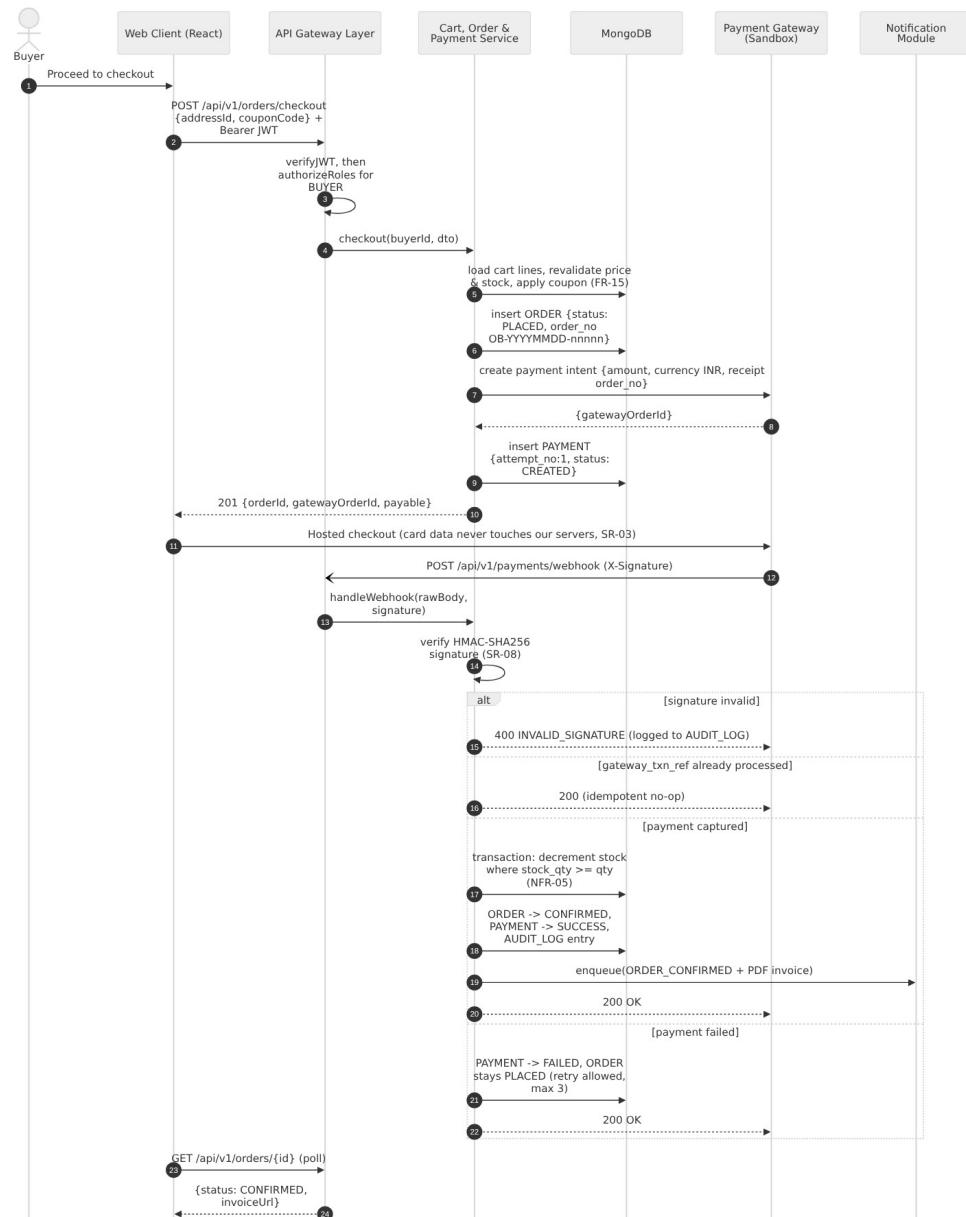


Figure 4: Checkout, payment and confirmation

4.3 API Design

All URLs start with /api/v1 and use JSON. Logged-in requests send the access token in the Authorization header.

Auth & User Service

Method and path	Who	Purpose	Main errors
POST /auth/register	Anyone	Create account, send OTP	400, 409 EMAIL_TAKEN

Method and path	Who	Purpose	Main errors
POST /auth/verify-otp	Anyone	Activate account	400 OTP_INVALID, 410 OTP_EXPIRED
POST /auth/login	Anyone	Get tokens	401 INVALID_CREDENTIALS, 429
POST /auth/refresh, /auth/logout	User	New token / sign out	401
POST /auth/forgot- password, /auth/reset- password	Anyone	Reset password	400 RESET_TOKEN_INVALID
GET, PATCH /users/me; /users/me/addresses	User / Buyer	Profile and addresses	400, 404, 409 ADDRESS_IN_USE

Catalogue & Search Service

Method and path	Who	Purpose	Main errors
GET /books? q=&category=&sort=&page=	Anyone	Search and filter books	400
GET /books/:id	Anyone	Book details with all offers	404 BOOK_NOT_FOUND
GET /books/isbn/:isbn13	Seller	Look up a book by ISBN	400 ISBN_INVALID, 404
POST /books/:id/reviews	Buyer	Review a delivered book	403 REVIEW_NOT_ELIGIBLE

Cart, Order & Payment Service

Method and path	Who	Purpose	Main errors
GET, POST, PATCH, DELETE /cart/items	Buyer	Manage cart	409 OUT_OF_STOCK
POST /orders/checkout	Buyer	Place order and start payment	409 PRICE_CHANGED, 422 COUPON_INVALID
POST /payments/webhook	Gateway	Payment result	400 INVALID_SIGNATURE
GET /orders/:id; POST /orders/:id/cancel	Buyer	Track or cancel	404, 409 ORDER_STATE_INVALID

Seller, Admin & Reporting Service

Method and path	Who	Purpose	Main errors
POST, PATCH /seller/listings	Seller	Create or edit a listing	422 PRICE_ABOVE_MRP
PATCH /seller/order- items/:id/status	Seller	Pack and ship	409 ORDER_STATE_INVALID
GET /seller/dashboard	Seller	Sales and settlement	400
POST /admin/:type/:id/block	Admin	Block a user, listing or review	400 REASON_REQUIRED

Method and path	Who	Purpose	Main errors
GET /admin/analytics? month=&format=	Admin	Monthly report (CSV/PDF)	400

4.4 Error Handling, Logging & Monitoring

Every error is returned in the same shape, so the website can show it next to the right field:

```
{ "error": { "code": "OUT_OF_STOCK", "message": "Only 2 copies are
available.", "requestId": "..."} }
```

- 400 bad input, 401 not logged in, 403 not allowed, 404 not found, 409 conflict, 429 too many requests, 500 server error.
- Unexpected errors never show internal details to the user.
- Logs are in JSON and never contain passwords, OTPs or tokens.
- An uptime monitor checks GET /health every five minutes.

4.5 UX Design

- Search box on every page; filters shown as removable chips.
- Checkout in at most four screens: Cart, Address, Pay, Confirmation.
- Works on phones down to 360 px wide; keyboard-friendly; good colour contrast.
- Errors shown next to the field that caused them.

4.6 Open Issues & Next Steps

- Pick the hosting provider after a free-tier test (Hardick, Sprint 1).
- Decide the tax rule shown on invoices (Pramod, Sprint 2).
- Next: build Sprint 1 features against the API above.

5. Appendices

5.1 Glossary

see SRS section 1.4 and Appendix A.

5.2 References

IEEE 42010, IEEE 1016, OWASP Top 10 (2021), SRS v1.1.

5.3 Tools

Graphviz and Mermaid (diagrams), Swagger (API), GitHub, Jira.